Assignment No. 1

Date: January 12, 2026

Title: Installation and Configuration of PostgreSQL and Client-Server Integration with Angular.

Objective / Aim: To install and configure the PostgreSQL database server, perform basic DDL and DML operations, and integrate the database with a web-based client using Node.js and Angular.

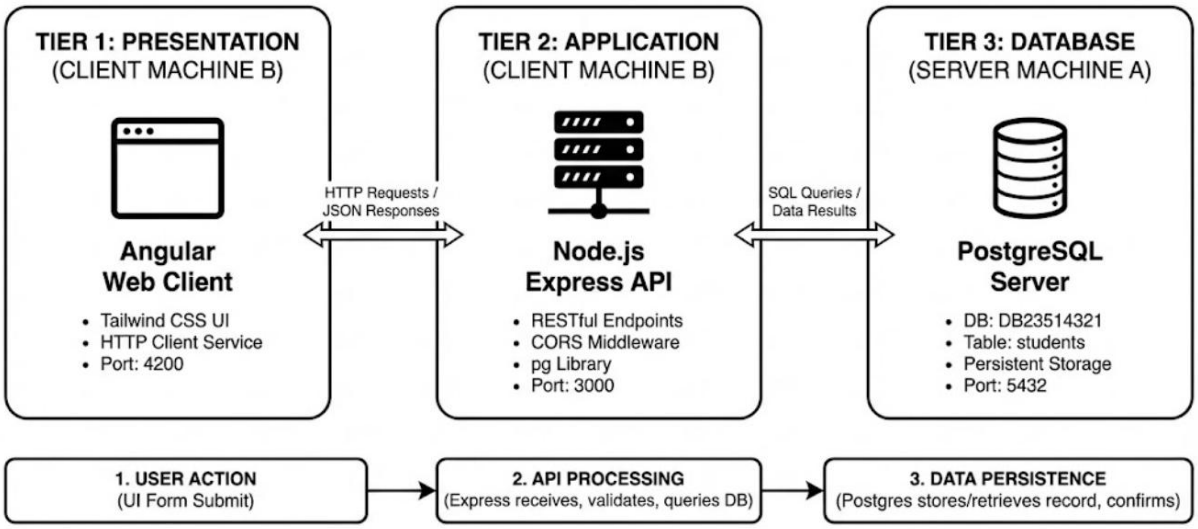
1. Introduction

This assignment demonstrates the implementation of a 3-tier architecture. It involves setting up a relational database (PostgreSQL), a server-side bridge (Node.js/Express), and a client-side interface (Angular). The project focuses on "CRUD" operations (Create, Read, Update, Delete) to manage student records effectively.

2. Theory / Algorithms

- **PostgreSQL:** An open-source object-relational database system known for reliability and data integrity.
- **3-Tier Architecture:** 1. **Database Tier:** PostgreSQL stores student records in a relational table. 2. **Application Tier:** Node.js acts as a "Bridge," using the pg library to execute SQL queries and cors to allow cross-origin requests. 3. **Presentation Tier:** Angular provides a reactive user interface styled with Tailwind CSS.
- **Algorithm for Data Flow:**
    1. User enters data in the Angular form.
    2. Angular DataService sends an HTTP POST request to the Node.js API.
    3. The Node.js server executes an INSERT query into PostgreSQL.
    4. Angular refreshes the view by fetching the updated list via an HTTP GET request.

3. Documentation: Functional Block Diagram



4. Procedure

- **Database Layer:** Installed PostgreSQL 18 and used pgAdmin 4 to create a new login role and database named **DB23510014**.
- **Schema Design:** Created a Students table with columns for id, name, and prn.
- **Backend Layer:** Initialized a Node.js project, installed express, pg, and cors, and configured the database connection pool.
- **Frontend Layer:** Created an Angular project, configured Tailwind CSS, and generated a DataService to handle API communication.
- **Integration:** Connected the layers by ensuring the Node.js server (Port 3000) and Angular dev server (Port 4200) were running simultaneously.

5. Actual Experiments & Results

SQL Script Executed (Part 2):

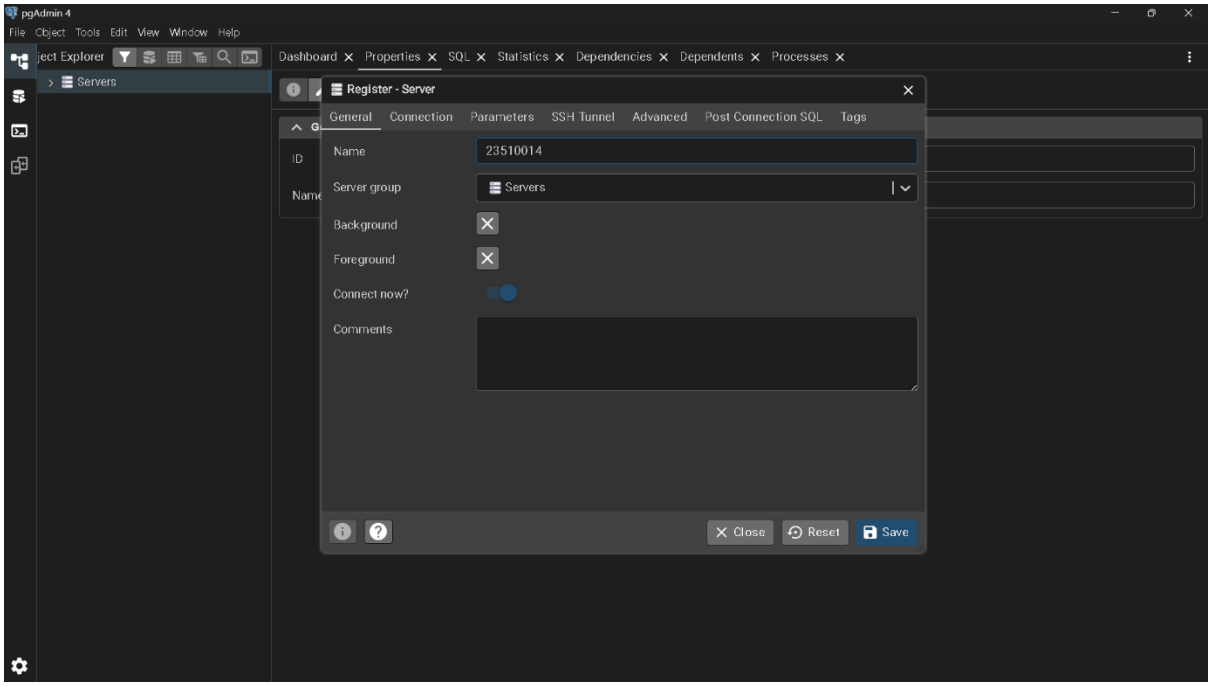
SQL

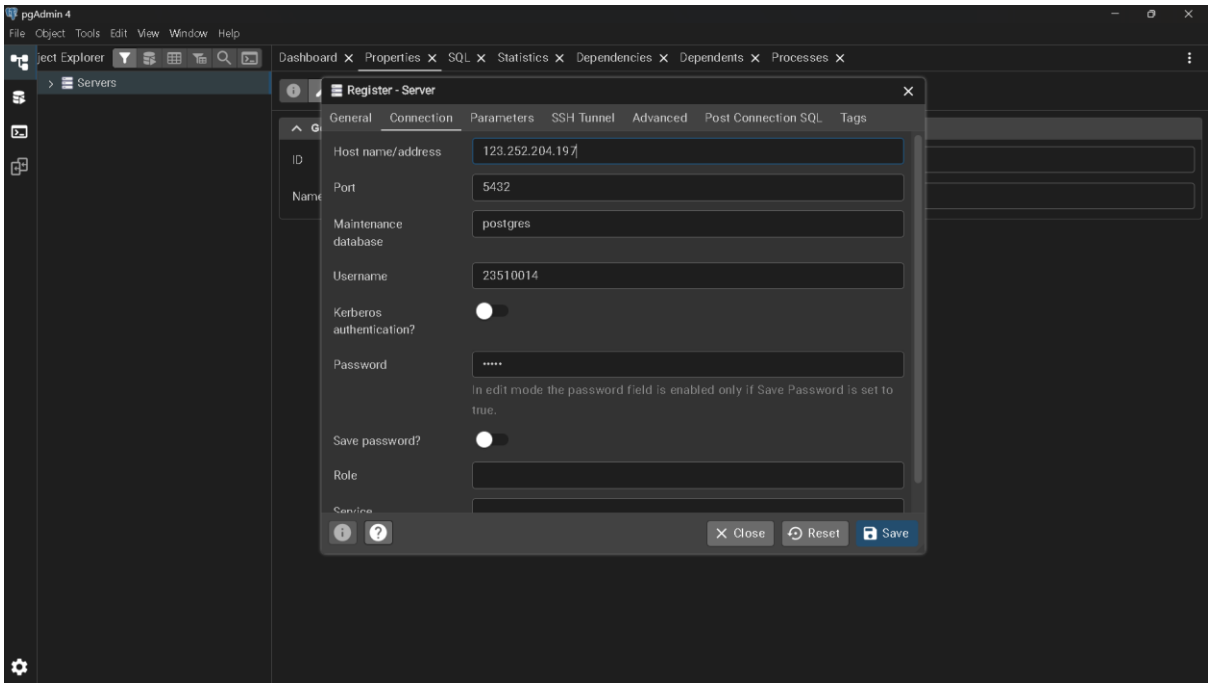
```
CREATE TABLE IF NOT EXISTS Student (

  id SERIAL PRIMARY KEY,
```

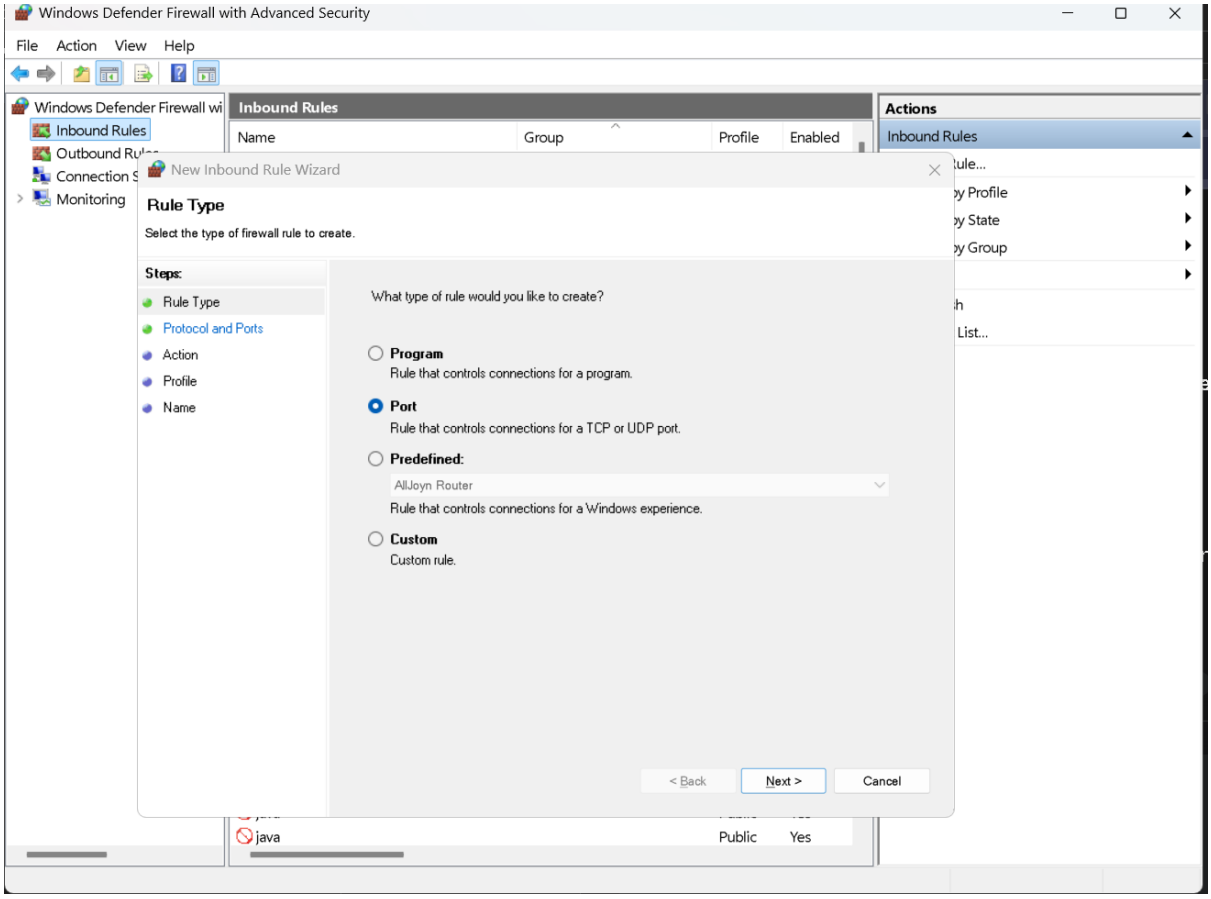
```
  name VARCHAR(100) NOT NULL,
  prn VARCHAR(20) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO Student (name, prn) VALUES
('MAYUR', '23510014'),
('ALICE', '23510015')
SELECT * FROM Students;
```

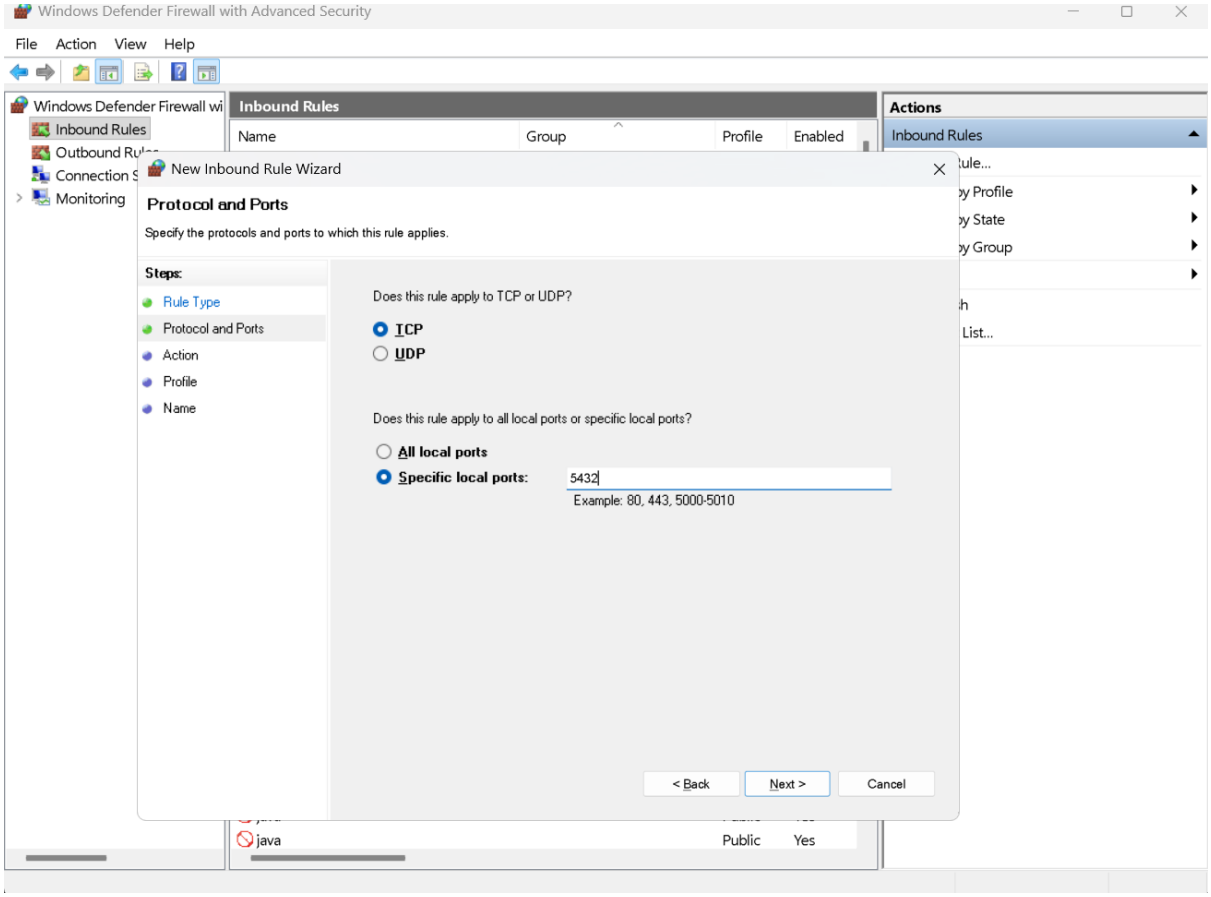
Observations:

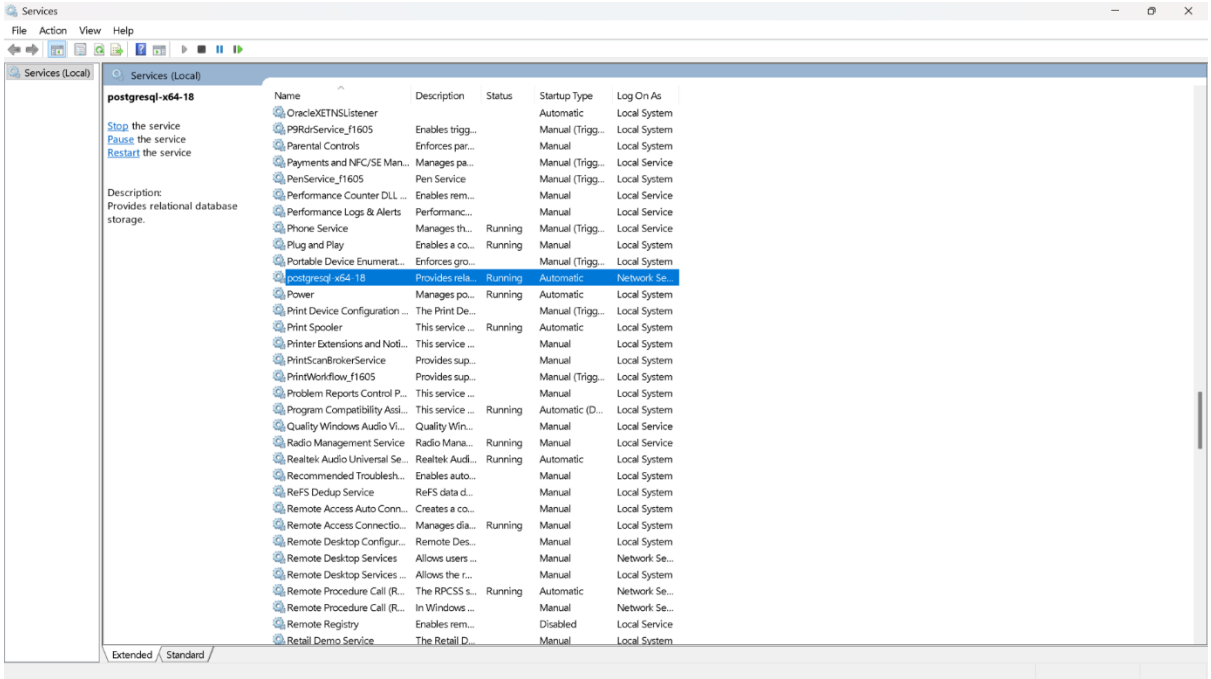
- The pgAdmin Query Tool successfully returned the record for **'MAYUR '** with the assigned ID.
- The Angular web client (localhost:4200) successfully rendered the student list in a Tailwind-styled table.
- The "Add Student" form successfully added **'RUPESH' (23510010)** to the database.

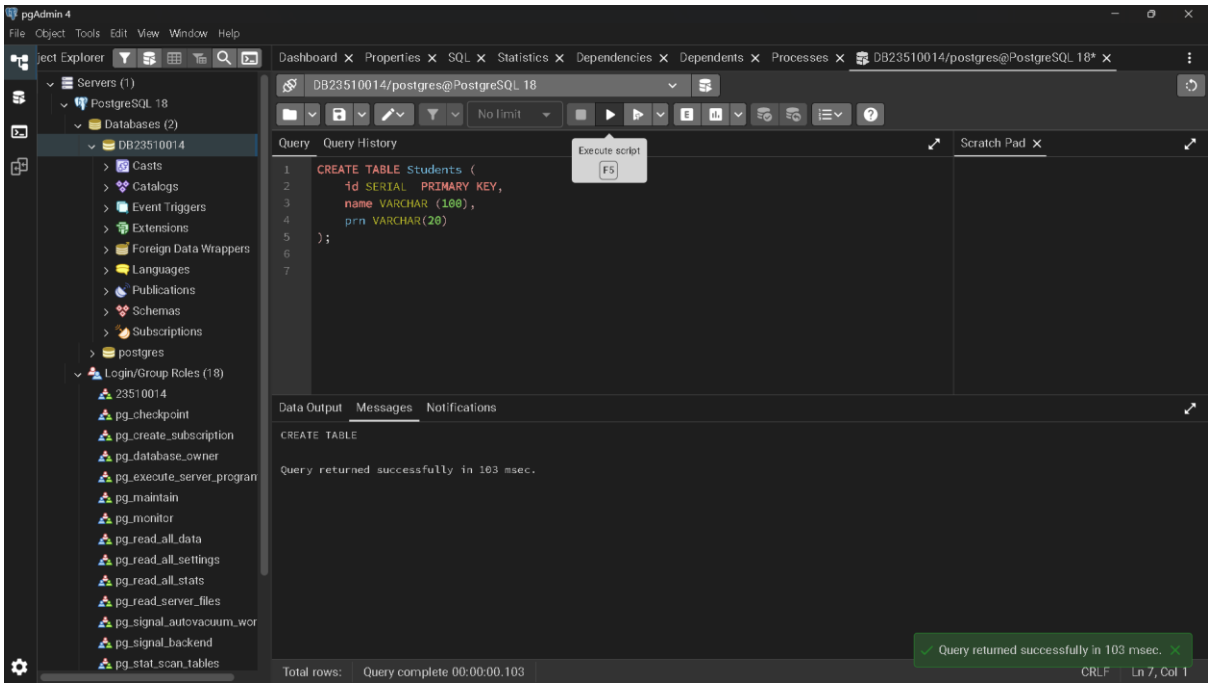

pgAdmin 4
File Object Tools Edit View Window Help
Servers
Dashboard Properties SQL Statistics Dependencies Dependents Processes
Register - Server
General Connection Parameters SSH Tunnel Advanced Post Connection SQL Tags
Name
23510014
Server group
Servers
Background
Foreground
Connect now?
Comments
Close
Reset
Save

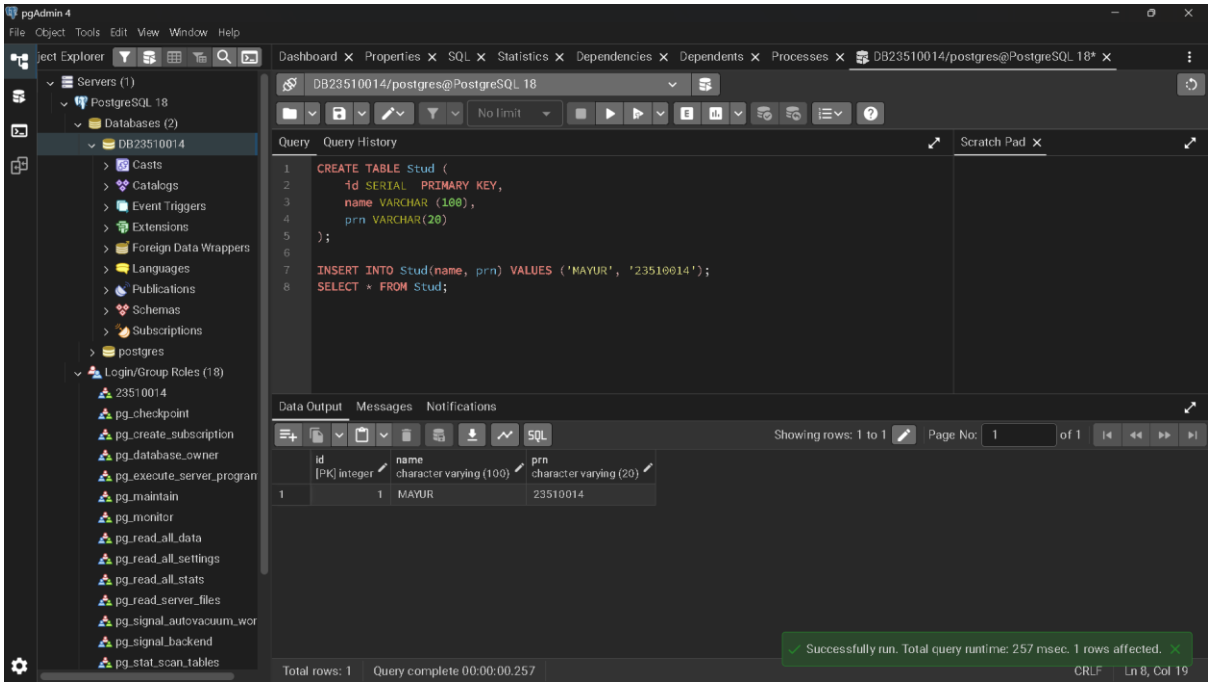

pgAdmin 4
File Object Tools Edit View Window Help
Servers
Dashboard Properties SQL Statistics Dependencies Dependents Processes
Register - Server
General Connection Parameters SSH Tunnel Advanced Post Connection SQL Tags
Host name/address
123.252.204.197
Port
5432
Maintenance database
postgres
Username
23510014
Kerberos authentication?
Password
In edit mode the password field is enabled only if Save Password is set to true.
Save password?
Role
Close
Reset
Save

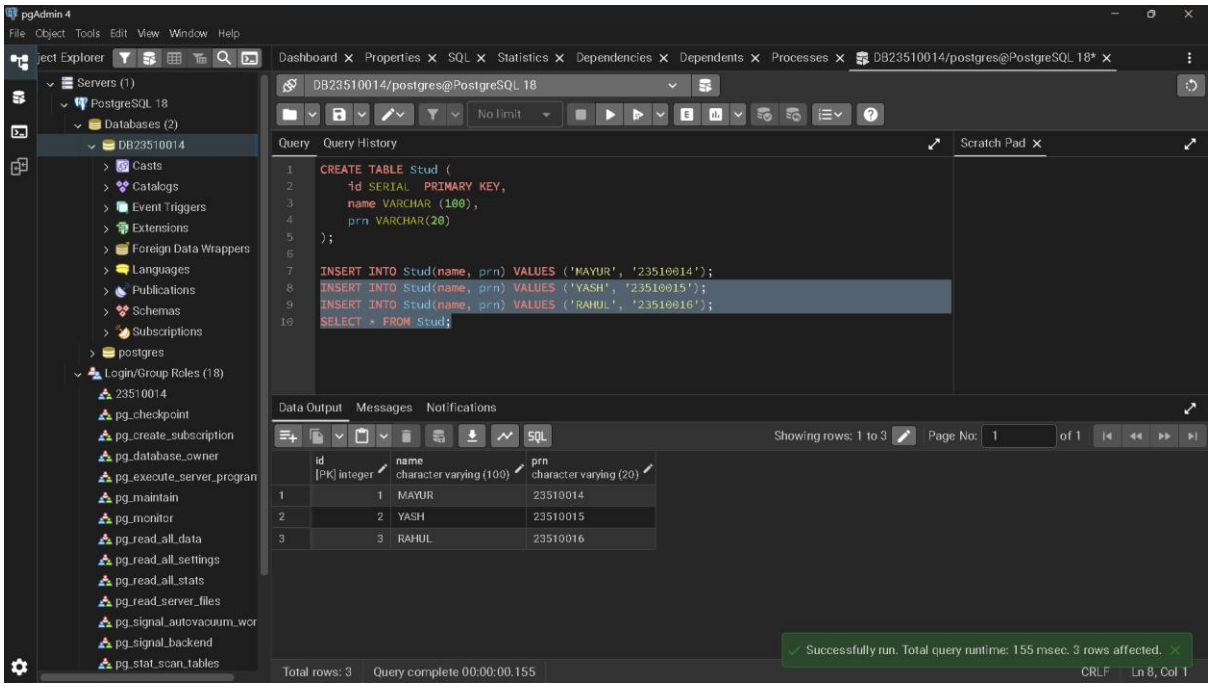

Windows Defender Firewall with Advanced Security
File Action View Help
Windows Defender Firewall wi
Inbound Rules
Outbound Ru
Connection S
Monitoring
Inbound Rules
Name
Group
Profile
Enabled
Actions
Inbound Rules
New Inbound Rule Wizard
Rule Type
Select the type of firewall rule to create.
Steps:
Rule Type
Protocol and Ports
Action
Profile
Name
What type of rule would you like to create?
Program
Rule that controls connections for a program.
Port
Rule that controls connections for a TCP or UDP port.
Predefined:
AllJoyn Router
Rule that controls connections for a Windows experience.
Custom
Custom rule.
< Back
Next >
Cancel
java
Public
Yes


Windows Defender Firewall with Advanced Security
File Action View Help
Windows Defender Firewall wi
Inbound Rules
Outbound Ru
Connection S
Monitoring
Inbound Rules
Name
Group
Profile
Enabled
Actions
Inbound Rules
New Inbound Rule Wizard
Protocol and Ports
Specify the protocols and ports to which this rule applies.
Steps:
Rule Type
Protocol and Ports
Action
Profile
Name
Does this rule apply to TCP or UDP?
TCP
UDP
Does this rule apply to all local ports or specific local ports?
All local ports
Specific local ports:
5432
Example: 80, 443, 5000-5010
< Back
Next >
Cancel
java
Public
Yes

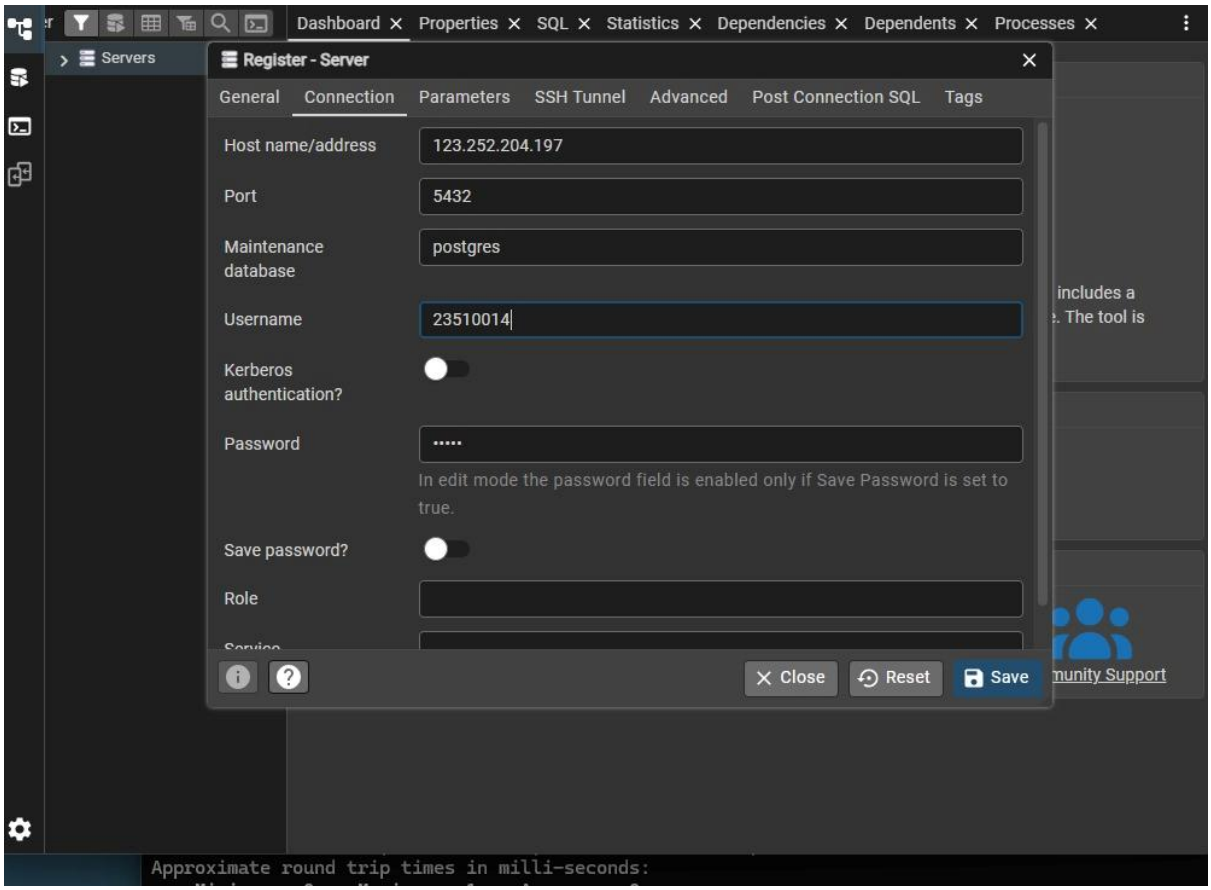

Services
File Action View Help
Services (Local)
postgresql-x64-18
Stop the service
Pause the service
Restart the service
Description:
Provides relational database storage.
Name Description Status Startup Type Log On As
OracleXETNSListener Automatic Local System
P9RdrService_f1605 Enables trigg... Manual (Trigg... Local System
Parental Controls Enforces par... Manual Local System
Payments and NFC/SE Man... Manages pa... Manual (Trigg... Local Service
PenService_f1605 Pen Service Manual (Trigg... Local System
Performance Counter DLL ... Enables rem... Manual Local Service
Performance Logs & Alerts Performanc... Manual Local Service
Phone Service Manages th... Running Manual (Trigg... Local Service
Plug and Play Enables a co... Running Manual Local System
Portable Device Enumerat... Enforces gro... Manual (Trigg... Local System
postgresql-x64-18 Provides rela... Running Automatic Network Se...
Power Manages po... Running Automatic Local System
Print Device Configuration ... The Print De... Manual (Trigg... Local System
Print Spooler This service ... Running Automatic Local System
Printer Extensions and Noti... This service ... Manual Local System
PrintScanBrokerService Provides sup... Manual Local System
PrintWorkflow_f1605 Provides sup... Manual (Trigg... Local System
Problem Reports Control P... This service ... Manual Local System
Program Compatibility Assi... This service ... Running Automatic (D... Local System
Quality Windows Audio Vi... Quality Win... Manual Local Service
Radio Management Service Radio Mana... Running Manual Local Service
Realtek Audio Universal Se... Realtek Audi... Running Automatic Local System
Recommended Troublesh... Enables auto... Manual Local System
ReFS Dedup Service ReFS data d... Manual Local System
Remote Access Auto Conn... Creates a co... Manual Local System
Remote Access Connectio... Manages dia... Running Manual Local System
Remote Desktop Configur... Remote Des... Manual Local System
Remote Desktop Services Allows users ... Manual Network Se...
Remote Desktop Services ... Allows the r... Manual Local System
Remote Procedure Call (R... The RPCSS s... Running Automatic Network Se...
Remote Procedure Call (R... In Windows ... Manual Network Se...
Remote Registry Enables rem... Disabled Local Service
Retail Demo Service The Retail D... Manual Local System
Extended Standard


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Processes DB23510014/postgres@PostgreSQL 18*
Servers (1)
PostgreSQL 18
Databases (2)
DB23510014
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas
Subscriptions
postgres
Login/Group Roles (18)
23510014
pg_checkpoint
pg_create_subscription
pg_database_owner
pg_execute_server_program
pg_maintain
pg_monitor
pg_read_all_data
pg_read_all_settings
pg_read_all_stats
pg_read_server_files
pg_signal_autovacuum_wor
pg_signal_backend
pg_stat_scan_tables
DB23510014/postgres@PostgreSQL 18
No limit
Query Query History
Execute script
F5
Scratch Pad
CREATE TABLE Students (
id SERIAL PRIMARY KEY,
name VARCHAR (100),
prn VARCHAR(20)
);
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 103 msec.
Total rows: Query complete 00:00:00.103
Query returned successfully in 103 msec.
CRLF Ln 7, Col 1

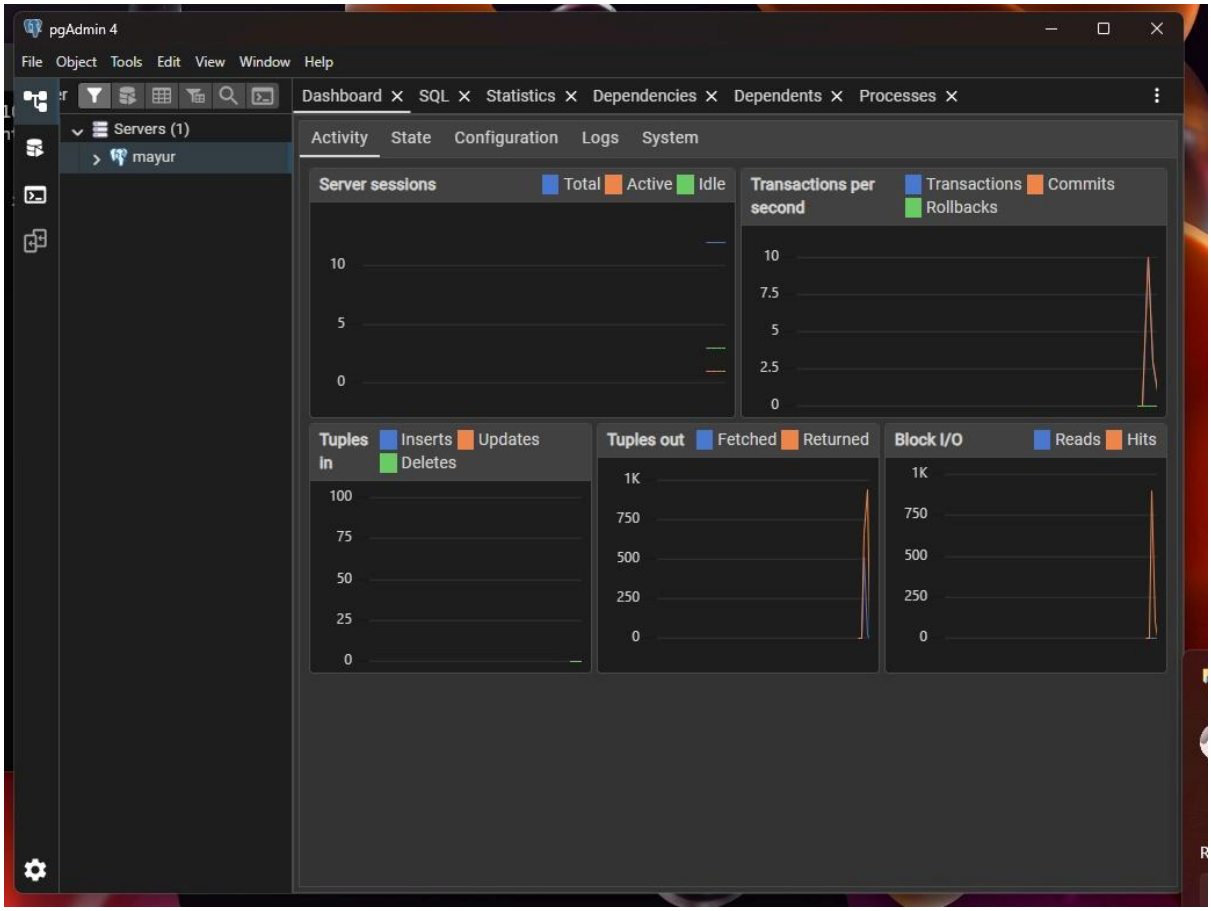

pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Processes DB23510014/postgres@PostgreSQL 18*
DB23510014/postgres@PostgreSQL 18
Query Query History
Scratch Pad
CREATE TABLE Stud (
id SERIAL PRIMARY KEY,
name VARCHAR (100),
prn VARCHAR(20)
);
INSERT INTO Stud(name, prn) VALUES ('MAYUR', '23510014');
SELECT * FROM Stud;
Data Output Messages Notifications
Showing rows: 1 to 1
Page No: 1 of 1
id [PK] integer
name character varying (100)
prn character varying (20)
1 1 MAYUR 23510014
Successfully run. Total query runtime: 257 msec. 1 rows affected.
Total rows: 1
Query complete 00:00:00.257
CRLF
Ln 8, Col 19


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Processes DB23510014/postgres@PostgreSQL 18*
DB23510014/postgres@PostgreSQL 18
Query Query History
Scratch Pad
CREATE TABLE Stud (
id SERIAL PRIMARY KEY,
name VARCHAR (100),
prn VARCHAR(20)
);
INSERT INTO Stud(name, prn) VALUES ('MAYUR', '23510014');
INSERT INTO Stud(name, prn) VALUES ('YASH', '23510015');
INSERT INTO Stud(name, prn) VALUES ('RAHUL', '23510016');
SELECT * FROM Stud;
Data Output Messages Notifications
Showing rows: 1 to 3
Page No: 1 of 1
id [PK] integer
name character varying (100)
prn character varying (20)
1 1 MAYUR 23510014
2 2 YASH 23510015
3 3 RAHUL 23510016
Successfully run. Total query runtime: 155 msec. 3 rows affected.
Total rows: 3
Query complete 00:00:00.155
CRLF
Ln 8, Col 1

```
C:\Users\mayur>ping 10.40.4.167

Pinging 10.40.4.167 with 32 bytes of data:
Reply from 10.40.4.167: bytes=32 time=4ms TTL=128
Reply from 10.40.4.167: bytes=32 time=5ms TTL=128
Reply from 10.40.4.167: bytes=32 time=4ms TTL=128
Reply from 10.40.4.167: bytes=32 time=5ms TTL=128

Ping statistics for 10.40.4.167:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 4ms, Maximum = 5ms, Average = 4ms
```



```
Command Prompt

    -6              Force using IPv6.


C:\Users\2731>ping 123.252.204.197

Pinging 123.252.204.197 with 32 bytes of data:
Reply from 123.252.204.197: bytes=32 time<1ms TTL=63
Reply from 123.252.204.197: bytes=32 time=1ms TTL=63
Reply from 123.252.204.197: bytes=32 time<1ms TTL=63
Reply from 123.252.204.197: bytes=32 time=1ms TTL=63

Ping statistics for 123.252.204.197:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\2731>ping 10.4.2.63

Pinging 10.4.2.63 with 32 bytes of data:
Reply from 10.4.2.63: bytes=32 time=3ms TTL=128
Reply from 10.4.2.63: bytes=32 time=188ms TTL=128
Reply from 10.4.2.63: bytes=32 time=94ms TTL=128
Reply from 10.4.2.63: bytes=32 time=103ms TTL=128

Ping statistics for 10.4.2.63:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 3ms, Maximum = 188ms, Average = 97ms

C:\Users\2731>telnet 10.4.2.63 5432
```



6. Conclusion

The installation and configuration of the PostgreSQL server were successfully completed. By integrating it with an Angular client via a Node.js bridge, we successfully demonstrated the capability to perform real-time CRUD operations in a 3-tier environment. The use of Tailwind CSS ensured a modern, responsive user interface.

7. References

- PostgreSQL Official Documentation: https://www.postgresql.org/docs/
- Angular Guide: https://angular.dev/
- Tailwind CSS Documentation: https://tailwindcss.com/docs/installation